PROJECT DOCUMENTATION

Automatic Street Light Controlling System Using Sensors and LDR at Night (ESP32 IoT Based)

LDR day / night detection, ultrasonic vehicle detection and energy-saving white / green LED street lighting

Parth Dalakoti

B.Tech Electronics & Communication Engineering (Final Year)
Graphic Era Hill University, Bhimtal Campus

October 2026

1. Project Overview

This project is an **automatic street light controller** that saves energy by lighting the road only as much as needed. An **LDR (light dependent resistor)** tells the controller whether it is day or night, and an **HC-SR04 ultrasonic sensor** detects a vehicle approaching. Two groups of LEDs represent the street lamps: **white LEDs** (bright lighting) and **green LEDs** (idle night lighting).

How it behaves

Situation	White LEDs (D3, D4, D5)	Green LEDs (D6, D7)
Daylight (LDR value above threshold)	OFF	OFF
Night, no vehicle nearby	OFF	ON (idle lighting)
Night, vehicle closer than 50 cm	ON	OFF
Night, 10 s after the last detection	OFF	ON

Key features

- Automatic **day / night** switching using an LDR on analog pin A0 (threshold 500).
- **Vehicle detection** with an HC-SR04 ultrasonic sensor (trigger distance below 50 cm).
- **Adaptive lighting**: bright white lights only when needed, low-power green idle lights otherwise.
- **Non-blocking hold timer**: white lights stay on for 10 s after the last detection, using `millis()` instead of `delay()`.
- **Serial monitor output** of the LDR value and distance (9600 baud) for calibration and debugging.
- Simple, low-cost hardware that can be extended to an **ESP32 IoT** version with Wi-Fi monitoring.

Contents of this document

Section	Topic
1	Project overview and behaviour
2	Specifications and components
3	System block diagram
4	Circuit diagram
5	Pin configuration and pin diagrams
6	Software design and working logic (flowchart)
7	Complete source code
8	Setup, calibration, troubleshooting, observations, ESP32 / IoT extension, conclusion

2. Specifications and Components

2.1 Working principle of the sensors

LDR: the resistance of an LDR falls as light increases. Connected in a voltage divider with a fixed resistor, it gives an analog voltage on A0. With the LDR on the +5V side and the fixed resistor to GND, a bright place gives a high reading and darkness gives a low reading, which matches the code (reading above 500 means day).

HC-SR04: a 10 µs HIGH pulse on TRIG makes the module send an ultrasonic burst. The ECHO pin stays HIGH for the time the sound takes to travel to the object and back. The code converts this time to distance with `distance = duration × 0.034 / 2` (speed of sound = 0.034 cm per µs, divided by 2 for the round trip). Example: an echo of 1470 µs gives about 25 cm; the 50 cm threshold corresponds to an echo of about 2941 µs.

2.2 Specifications (from the code)

Parameter	Value
Controller pins	Arduino-style numbering: A0, D3 to D7, D9, D10 (see note in section 8)
Supply	5 V
Day / night sensor	LDR on A0, 10-bit ADC (0 to 1023), threshold `ldrThreshold = 500`
Vehicle sensor	HC-SR04, TRIG = D9, ECHO = D10, detects when 0 < distance < `distanceThreshold` = 50 cm
White LEDs	3 LEDs on D3, D4, D5 (bright lamps)
Green LEDs	2 LEDs on D6, D7 (idle lamps)
White hold time	10 s (10000 ms) after the last detection
Loop delay	100 ms
Serial monitor	9600 baud: prints *LDR Value* and *Distance*

2.3 Components used

Qty	Component	Role in the project
1	Arduino Uno / Nano (see ESP32 note in section 8)	Main controller
1	LDR (photoresistor)	Detects daylight / darkness
1	Resistor 10 kΩ (typical)	Fixed resistor of the LDR voltage divider
1	HC-SR04 ultrasonic sensor	Measures distance to detect a vehicle
3	White LEDs	Bright street lamps
2	Green LEDs	Idle street lamps
5	Resistors 220 Ω (typical)	Current limiting for each LED
-	Breadboard, jumper wires, 5 V USB supply, model road / base	Assembly and power

Resistor values and the board are not written in the code, so the 10 kΩ and 220 Ω values are typical choices. Replace them with the values used in your actual build.

3. System Block Diagram

The LDR and the ultrasonic sensor feed the controller. The controller decides which LED group to switch on.

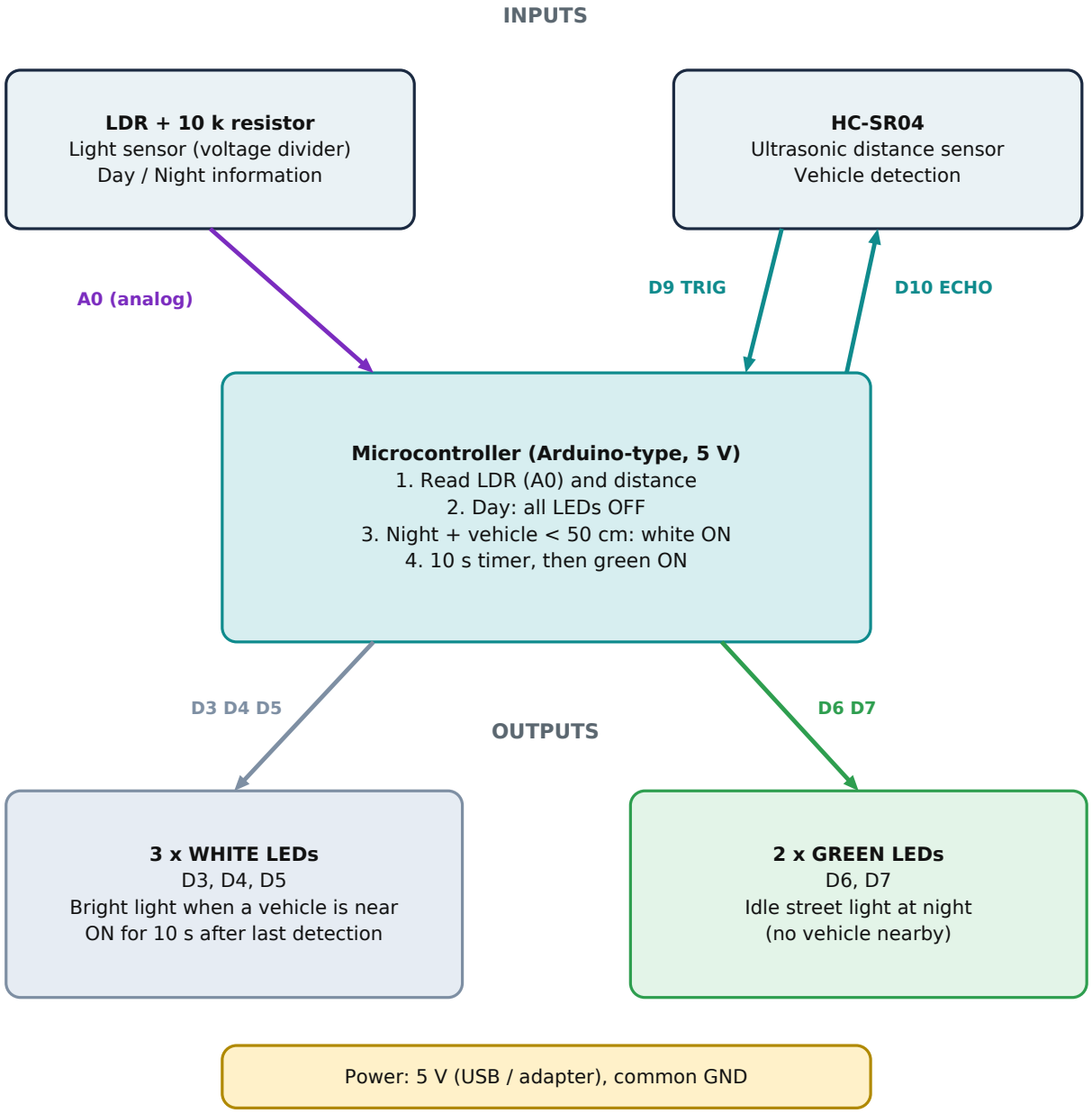
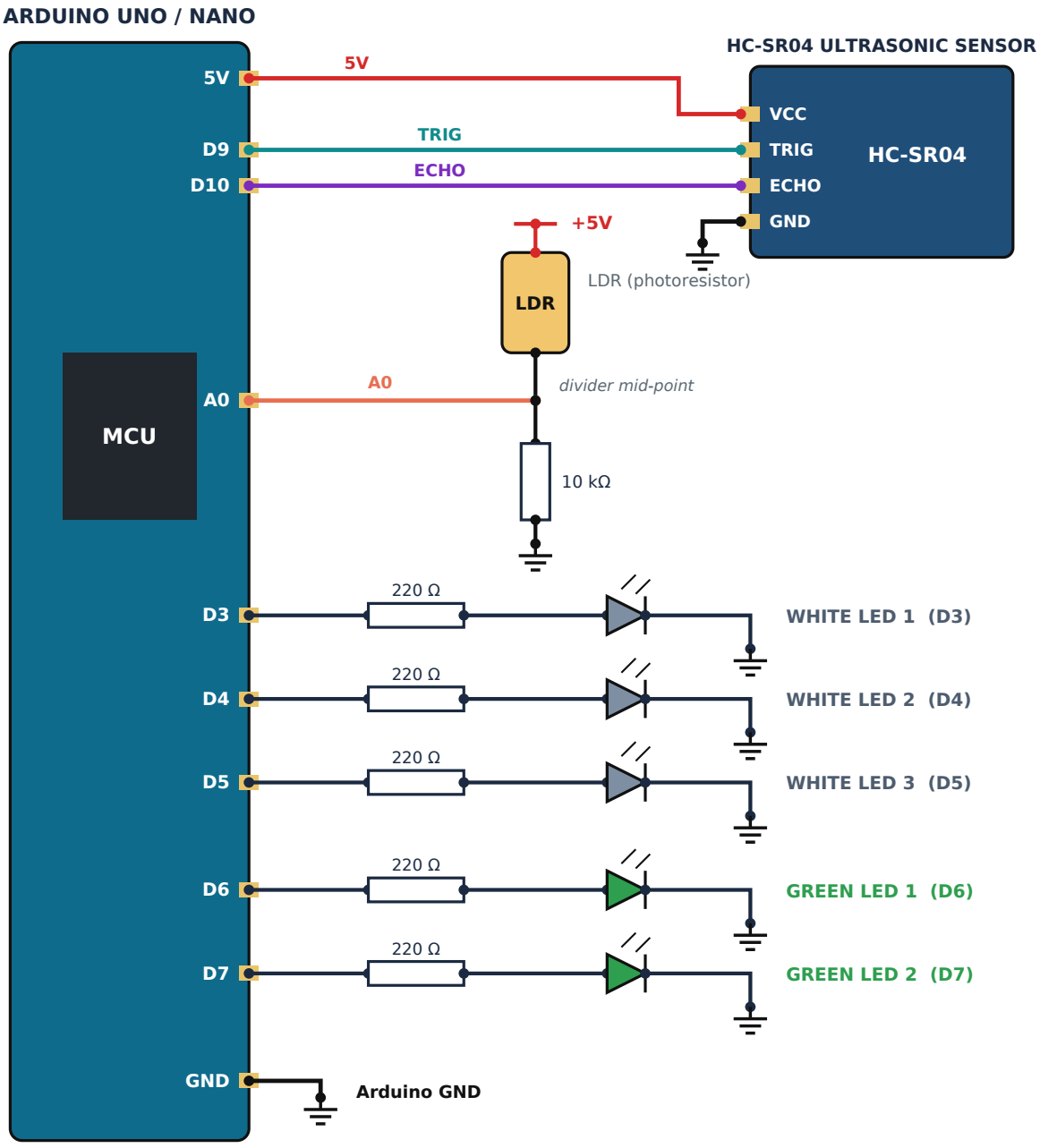


Figure 1: System block diagram

4. Circuit Diagram



All ground symbols are joined together (common GND with the Arduino GND pin).

LDR divider: LDR to +5V and 10 kΩ to GND, so a bright room gives a higher A0 value (code: day if > 500).

Every LED has its own series resistor (220 Ω typical). Anode towards the resistor, cathode to GND.

Figure 2: Complete circuit: LDR voltage divider on A0, HC-SR04 on D9 / D10, three white LEDs on D3 to D5 and two green LEDs on D6 and D7, each LED with its own series resistor.

5. Pin Configuration

5.1 Controller pin connections

Pin	Name in code	Connected to	Type / function
A0	LDR_PIN	LDR and 10 kΩ divider mid-point	Analog input, 0 to 1023
D9	TRIG_PIN	HC-SR04 TRIG	Digital output, 10 µs trigger pulse
D10	ECHO_PIN	HC-SR04 ECHO	Digital input, measured with `pulseIn()`
D3	WHITE_LED1	White LED 1 (through 220 Ω)	Digital output
D4	WHITE_LED2	White LED 2 (through 220 Ω)	Digital output
D5	WHITE_LED3	White LED 3 (through 220 Ω)	Digital output
D6	GREEN_LED1	Green LED 1 (through 220 Ω)	Digital output
D7	GREEN_LED2	Green LED 2 (through 220 Ω)	Digital output
5V	-	HC-SR04 VCC and top of LDR	Supply
GND	-	HC-SR04 GND, 10 kΩ resistor, all LED cathodes	Common ground

5.2 HC-SR04 module pins

HC-SR04 pin	Connected to	Function
VCC	5V	Power supply
TRIG	D9	Trigger input, a 10 µs HIGH pulse starts a measurement
ECHO	D10	Echo output, HIGH for the round-trip time of the sound
GND	GND	Ground

5.3 LDR divider and LED wiring

Part	Connection
LDR	One leg to +5V, other leg to the divider mid-point (A0)
10 kΩ resistor	Between the divider mid-point (A0) and GND
Each LED	Controller pin, then 220 Ω resistor, then LED anode (long leg); cathode (short leg) to GND

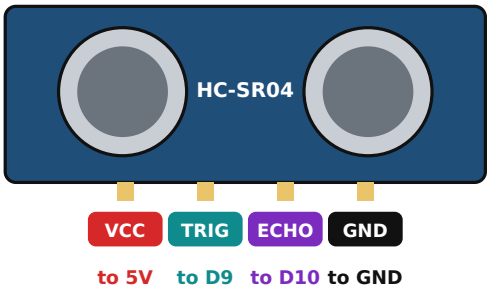
Current check: only one LED group is on at a time (3 white or 2 green), so the total LED current stays well within the board limits (each LED at roughly 10 to 20 mA).

5.4 Pin diagram

Arduino Uno R3 header layout with the used pins highlighted. An Arduino Nano uses the same pin numbers (D3 to D10, A0, 5V, GND).



HC-SR04 ultrasonic sensor module pinout



Trigger: 10 us HIGH pulse on TRIG. Echo pulse width on ECHO = round-trip time of the ultrasonic burst.



Figure 3: Pin usage on the controller and pinout of the HC-SR04 module

6. Software Design

6.1 Functions in the sketch

Function	What it does
setup()	Starts Serial at 9600 baud, sets TRIG and the five LED pins as outputs, ECHO as input, and switches all lights off
getDistance()	Sends a 10 µs pulse on TRIG, measures the ECHO pulse with `pulseIn()` and returns distance in cm
allLightsOff()	Turns off all three white and both green LEDs
whiteLightsOn()	Turns on the three white LEDs
greenLightsOn() / greenLightsOff()	Turns the two green LEDs on / off
loop()	Reads sensors, prints them, then runs day mode or night mode logic

6.2 Variables and constants

Name	Value	Meaning
ldrThreshold	500	Reading above this is treated as daylight
distanceThreshold	50	Vehicle detected if distance is below 50 cm
whiteLedTimer	millis()	Time of the last vehicle detection
whiteLedState	true / false	True while the white lights are in their 10 s hold period

6.3 Mode logic

- **Day mode:** if `ldrValue > 500`, all LEDs are switched off, `whiteLedState` is cleared and the loop restarts after 100 ms.
- **Vehicle detected (night):** if `0 < distance < 50`, white LEDs turn on, green LEDs turn off, the timer is restarted and `whiteLedState` becomes true.
- **Hold period:** while `whiteLedState` is true, the white LEDs stay on. When `millis() - whiteLedTimer` reaches 10000 ms, white turns off and green turns on.
- **Idle night:** when no vehicle is being held, the green LEDs stay on.
- A new detection during the hold period restarts the 10 s timer, so the white lights stay on while traffic keeps passing.

6.4 Working logic (flowchart)

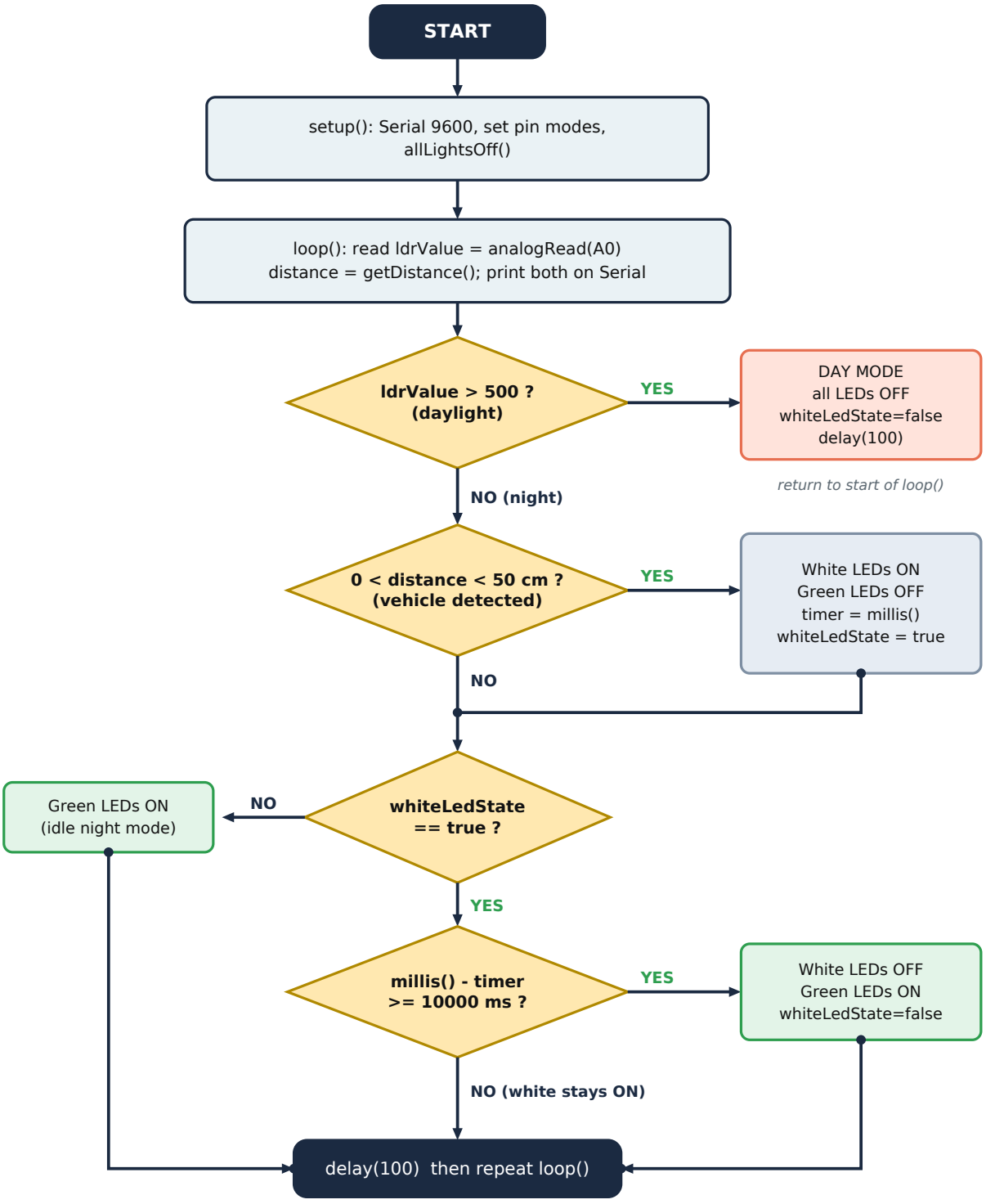


Figure 4: Flowchart of setup() and loop()

7. Complete Source Code

Language: Arduino C++. No external libraries are needed. Serial baud rate: 9600.

```
#define TRIG_PIN 9
#define ECHO_PIN 10
#define LDR_PIN A0

#define WHITE_LED1 3
#define WHITE_LED2 4
#define WHITE_LED3 5

#define GREEN_LED1 6
#define GREEN_LED2 7

const int ldrThreshold = 500;      // Adjust according to your LDR readings
const int distanceThreshold = 50;  // Vehicle detection range in cm

unsigned long whiteLedTimer = 0;
bool whiteLedState = false;

void setup() {
  Serial.begin(9600);

  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);

  pinMode(WHITE_LED1, OUTPUT);
  pinMode(WHITE_LED2, OUTPUT);
  pinMode(WHITE_LED3, OUTPUT);

  pinMode(GREEN_LED1, OUTPUT);
  pinMode(GREEN_LED2, OUTPUT);

  allLightsOff();
}

long getDistance() {
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);

  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);

  long duration = pulseIn(ECHO_PIN, HIGH);
  long distance = duration * 0.034 / 2;
  return distance;
}

void allLightsOff() {
  digitalWrite(WHITE_LED1, LOW);
  digitalWrite(WHITE_LED2, LOW);
  digitalWrite(WHITE_LED3, LOW);
  digitalWrite(GREEN_LED1, LOW);
  digitalWrite(GREEN_LED2, LOW);
}

void whiteLightsOn() {
  digitalWrite(WHITE_LED1, HIGH);
  digitalWrite(WHITE_LED2, HIGH);
  digitalWrite(WHITE_LED3, HIGH);
}

void greenLightsOn() {
  digitalWrite(GREEN_LED1, HIGH);
  digitalWrite(GREEN_LED2, HIGH);
}

void greenLightsOff() {
  digitalWrite(GREEN_LED1, LOW);
  digitalWrite(GREEN_LED2, LOW);
}

void loop() {
  int ldrValue = analogRead(LDR_PIN);
  long distance = getDistance();
```

```
 74
 75    Serial.print("LDR Value: ");
 76    Serial.print(ldrValue);
 77    Serial.print(" | Distance: ");
 78    Serial.println(distance);
 79
 80    // DAY MODE
 81    if (ldrValue > ldrThreshold) {
 82      allLightsOff();
 83      whiteLedState = false;
 84      delay(100);
 85      return;
 86    }
 87
 88    // NIGHT MODE
 89    if (distance > 0 && distance < distanceThreshold) {
 90      whiteLightsOn();
 91      greenLightsOff();
 92      whiteLedTimer = millis();
 93      whiteLedState = true;
 94    }
 95
 96    if (whiteLedState) {
 97      if (millis() - whiteLedTimer >= 10000) {
 98        digitalWrite(WHITE_LED1, LOW);
 99        digitalWrite(WHITE_LED2, LOW);
100        digitalWrite(WHITE_LED3, LOW);
101
102        greenLightsOn();
103        whiteLedState = false;
104      }
105    } else {
106      greenLightsOn();
107    }
108
109    delay(100);
110  }
```

8. Setup, Calibration and Troubleshooting

8.1 Setup and calibration

1. Wire the circuit as in Figure 2 and check that all grounds are common.
2. Upload the sketch and open the Serial Monitor at 9600 baud.
3. Cover the LDR with your hand and note the *LDR Value* (dark). Then expose it to normal room light and note the value (bright).
4. Set `ldrThreshold` roughly between the dark and bright readings. If your readings are reversed (dark gives a higher value), swap the LDR and the fixed resistor or invert the comparison.
5. Place a hand or toy vehicle in front of the HC-SR04 and check that *Distance* drops below 50 when it is close. Change `distanceThreshold` to suit the model road.
6. In the dark, confirm green is on when idle, white is on for about 10 s after a detection, and everything is off in bright light.

8.2 Troubleshooting

Problem	Likely cause	Fix
Lights never turn on	LDR always reads above 500	Check divider wiring; adjust `ldrThreshold` using Serial values
Always day or always night	Divider reversed or wrong threshold	Compare readings in light and dark and set the threshold between them
Distance always 0	No echo received, wrong TRIG / ECHO pins, or no 5 V on the sensor	Check D9 / D10 wiring and VCC / GND
Response is slow when nothing is in front	`pulseIn()` waits for its default timeout	Add a timeout, see section 8.3
LED dim or very bright	Missing or wrong series resistor	Use a 220 Ω resistor in series with every LED
White LEDs flicker at the 50 cm limit	Object right at the threshold	Move the object, or add a small margin or hysteresis

8.3 Observations and possible improvements

- **Board vs title:** the title says ESP32, but this sketch uses Arduino-style pins (`A0`, D3 to D10) and 10-bit ADC values. It is written for an Arduino Uno / Nano and will not work on an ESP32 without changes (see section 8.4).
- **Blocking `pulseIn()`:** with no timeout it can wait up to 1 s when nothing reflects the sound, which slows the loop. Use `pulseIn(ECHO_PIN, HIGH, 30000)` to limit the wait to about 5 m.
- **Hysteresis:** one threshold (500) can make the lights flicker at dusk. Use two thresholds, for example ON below 450 and OFF above 550.
- **Sensor in day mode:** the ultrasonic sensor is still read in day mode; skipping it would save a little power.
- **No IoT code:** the supplied sketch is fully local. Wi-Fi monitoring needs an ESP32 version (below).

8.4 Porting to ESP32 and IoT extension (suggestion)

The pin numbers below are a suggested mapping and are not part of the supplied code. They are not tested on hardware.

Function	Arduino sketch	Suggested ESP32 GPIO	Remark
LDR input	A0	GPIO34 (ADC1)	Input-only pin, 12-bit (0 to 4095): scale the threshold about 4x (about 2000). Power the divider from 3V3
HC-SR04 TRIG	D9	GPIO23	Output
HC-SR04 ECHO	D10	GPIO22	ECHO is 5 V: use a resistor divider (for example 1 kΩ + 2 kΩ) to bring it to 3.3 V
White LEDs	D3, D4, D5	GPIO25, 26, 27	Series resistors as before
Green LEDs	D6, D7	GPIO32, 33	Series resistors as before

GPIO6 to GPIO11 are used by the flash memory on most ESP32 boards, so D9 and D10 of the Arduino sketch cannot simply be reused there. For the IoT part, the ESP32 can connect to Wi-Fi and publish the LDR value, detected vehicle count and lamp state to a dashboard (for example MQTT, ThingSpeak or Blynk).

9. Conclusion

The project demonstrates an energy-saving street light that reacts to ambient light and to approaching vehicles. Day and night are separated by an LDR, vehicles are found by an ultrasonic sensor, and a millis()-based timer keeps the white lights on for 10 s after the last vehicle. The design is simple and low cost, and it can be extended with an ESP32 for Wi-Fi monitoring, multiple lamp poles and data logging.